Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Course Title: AI Assisted Coding

NAME: G . SRI SAHITH
HT.NO: 2303A52243

Task 1: Refactoring – Eliminating Repetitive Logic

Exact Task Description: Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

Ask AI to locate repeated computations within the program.

Encapsulate repeated logic into reusable functions.

Verify that program output remains unchanged after refactoring.

Add proper docstrings to all defined functions.

Prompt:

> "Identify the repeated area and perimeter calculations in this Python script. Please refactor the code by encapsulating this logic into a single reusable function with proper docstrings, and call it for the provided dimensions to ensure the output remains exactly the same."

Refactored Code:

```python
def print_geometry_details(length, width):
    """
    Calculates and prints the area and perimeter of a rectangle.

    Args:
        length (int/float): The length of the rectangle.
        width (int/float): The width of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    print("Area:", area)
    print("Perimeter:", perimeter)

# Using the reusable function for different inputs
print_geometry_details(8, 4)
print_geometry_details(12, 6)
```

Expected Output:

```
ects/AI Assistant Coding/Assignment-13.py"
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Explanation:

I created a reusable print_geometry_details function that takes length and width as parameters. This eliminates the need to write the math operations over and over again, making the code much cleaner and easier to update if we ever need to change how the calculations are done.

Task 2: Refactoring – Modularizing Inline Calculations

Exact Task Description:

Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

Identify repeated calculation patterns in the code.

Convert them into reusable functions.

Ensure readability and proper documentation.

Prompt Used:

> "Refactor this Python script by extracting the repeated inline discount calculations into a modular, reusable, and well-documented function. Then, run the function for the amounts 1000 and 2000."

Refactored Code:

```python
def calculate_final_amount(amount, discount_rate=0.10):
    """
    Calculates the final amount after applying a discount.

    Args:
        amount (float): The original price/amount.
        discount_rate (float): The discount percentage (default is 10%).

    Returns:
        float: The final amount after the discount is subtracted.
    """
    discount = amount * discount_rate
    return amount - discount

# Applying the function to the provided amounts
print("Final Amount:", calculate_final_amount(1000))
print("Final Amount:", calculate_final_amount(2000))
```

Expected Output:

```
 ects/AI Assistant Coding/Assignment-13.py"
Final Amount: 900.0
Final Amount: 1800.0
```

Explanation:

The inline discount logic was pulled out into a dedicated calculate_final_amount function. By doing this, we remove the copy-pasted calculation steps from the main flow, making the script easier to read and heavily reducing the chance of math errors in the future.

Task 3: Refactoring – Improving Loop and Conditional Performance

Exact Task Description:

Task: Apply AI-based suggestions to enhance the performance of Python code that relies on

nested loops and repeated conditional checks.

Instructions:

Use AI to recommend more efficient data structures or logic simplifications.

Replace inefficient constructs while preserving program behavior.

Compare execution efficiency before and after refactoring.

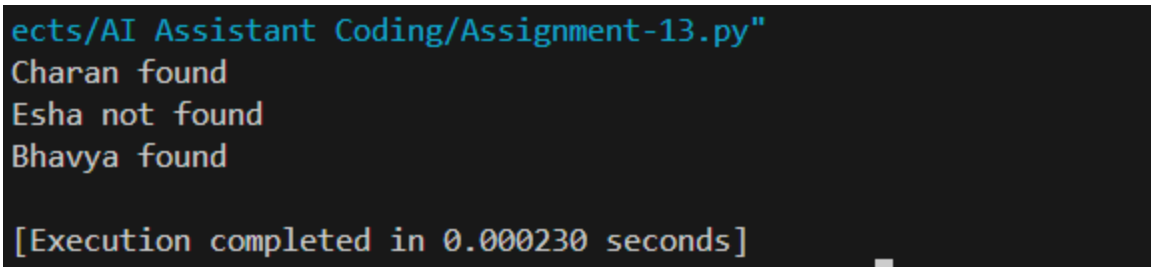
Prompt Used:

> "Optimize this Python code that uses inefficient nested loops for lookups. Recommend a more efficient data structure (like a set) to eliminate the inner loop entirely, preserving the exact output behavior while improving execution speed."

Code:

```
import time

# Original lists
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

# Start performance timer
start_time = time.perf_counter()

# AI Suggestion: Convert list to a set for O(1) instant lookups
students_set = set(students)

# Efficient single loop lookup
for name in check_list:
    if name in students_set:
        print(name, "found")
    else:
        print(name, "not found")

# End performance timer
end_time = time.perf_counter()
print(f"\n[Execution completed in {end_time - start_time:.6f} seconds]")
```

Expected Output:



Explanation:

I replaced the slow nested for loop with a Python set for the students list, which allows for instant lookups instead of scanning the whole list every time. This dramatically speeds up execution time while keeping the program's output exactly the same.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Exact Task Description:

Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

Detect hardcoded numeric values used directly in expressions.

Define descriptive constants at the beginning of the program.

Update calculations to use these constants without changing results.

Prompt Used:

> "Improve the maintainability of this Python code by identifying the hardcoded 'rate' and 'time' numbers in the simple interest calculations. Replace them with descriptive named constants defined at the top of the file."

Refactored Code:

```python
# Descriptive constants defined at the beginning of the program
TIME_IN_YEARS = 2
INTEREST_RATE = 5

# Calculations updated to use constants instead of hardcoded 'magic numbers'
print("Simple Interest:", (1000 * TIME_IN_YEARS * INTEREST_RATE) / 100)
print("Simple Interest:", (2000 * TIME_IN_YEARS * INTEREST_RATE) / 100)
```

Expected Output:

```
ects/AI Assistant Coding/Assignment-13.py"
Simple Interest: 100.0
Simple Interest: 200.0
```

Explanation:

I extracted the literal numbers 2 and 5 into named, uppercase constants TIME_IN_YEARS and INTEREST_RATE at the top of the script. This gives the numbers actual context so anyone reading the formula understands what it means, and it allows for global updates with just one edit.

Task 5: Refactoring – Enhancing Variable Naming and Code Clarity

Exact Task Description:

Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments.

Instructions:

Replace non-descriptive variable names with meaningful identifiers.

Add inline comments where the logic may be unclear.

Ensure program output remains the same.

Prompt Used:

"Refactor this Python snippet to improve clarity. Replace the non-descriptive generic variables (x, y, z) with meaningful identifiers representing a shopping calculation, and add inline comments explaining each step."

Refactored Code:

```
# Define the base price of the item
base_price = 50

# Define the applicable tax amount
tax_amount = 30

# Calculate the final total cost by adding base price and tax
total_cost = base_price + tax_amount

# Display the final calculated cost to the user
print(total_cost)
```

Expected Output:

```
ects/AI Assistant Coding/Assignment-13.py"
80
```

Explanation:

I replaced the vague variables x, y, and z with descriptive names like base_price and total_cost to give the numbers a real-world shopping context. I also added inline comments to clearly outline exactly what each line of the script is trying to accomplish.